
EDA Bench: Testing Whether Language-Model Agents Build Functioning PCBs

Anonymous Author(s)

Affiliation

Address

email

Abstract

Can a language-model agent reconstruct a PCB that behaves correctly at its connectors? We introduce EDA Bench, a benchmark with 35 KiCad tasks derived from public open-hardware projects. A system receives a prompt and a standard KiCad environment and must produce a complete project. The grader then matches the submitted connector pins to the pins named by the task, traces the routed copper, applies deterministic test signals, runs ngspice where applicable, and checks KiCad design-rule output. It scores observable behavior at the board boundary rather than text overlap, screenshots, component counts, or reference-designator names. All 35 frozen human-designed references score 1.0, and all 35 severe structural-failure controls score at most 0.15. A broader fault sweep exposes both useful sensitivity and blind spots: swapped connector pins score 0.672 on average, while removing one passive component scores 0.987 and often goes undetected. Current systems are far from solving the benchmark. The strongest of six web-enabled harness configurations scores 4.58% overall; another builds all 35 projects but scores only 2.79%. Task-bootstrap and alternative weighting analyses preserve the low absolute performance but do not replace stochastic reruns. EDA Bench therefore measures a specific, inspectable gap between producing a parseable KiCad project and reproducing its required external electrical behavior.

1 Introduction

Language-model agents can edit files, run tools, inspect failures, and try again. Software benchmarks therefore judge the resulting program by running tests rather than comparing its text with a reference. SWE-bench, OSWorld, and ProgramBench are examples of this approach [Jimenez et al., 2023, Xie et al., 2024, Yang et al., 2026]. We apply the same basic idea to PCB reconstruction.

Hardware design has an analogous workflow. Engineers search for parts, inspect datasheets, draft schematics, route boards, run ERC and DRC checks, simulate where possible, and produce manufacturing files. For electronic design automation, however, execution-based testing is substantially harder. Correctness depends on routed connectivity, electrical behavior, manufacturability constraints, and external interfaces rather than only on executable program behavior. Existing CAD and EDA benchmarks cover important parts of this space, including text-to-CAD generation [Khan et al., 2024], geometric CAD validation [Epoch AI, 2026, CAD Arena, 2026, Barkley et al., 2026], schematic generation [Al Hasan et al., 2026, Zou et al., 2026, Luo et al., 2026], schematic understanding [Lu et al., 2026], board-level schematic design [Qiu et al., 2026], hardware bug repair [Cui et al., 2026], and PCB placement and routing reasoning [Li et al., 2026]. These evaluations motivate execution-based assessment for engineering artifacts, but they do not directly test whether an agent can reconstruct a routed KiCad PCB that preserves the target board’s external electrical behavior.

EDA Bench evaluates complete agent systems on source-backed KiCad reconstruction tasks. A system receives a fixed prompt and runtime files, then writes a KiCad project to `/workspace/final_project`. KiCad provides open-source schematic, simulation, layout, rendering, and manufacturing tools [KiCad Project, 2026]. The tasks come from public hardware projects, and the main baselines can use the web. This makes the task auditable but also allows source discovery; we report that access policy as part of the evaluated system and do not claim closed-book circuit invention.

The scoring rule is easiest to state as an external-I/O test. For each task, a grader-side file lists the board pins visible to another device, the test signals to apply, the values to measure, and acceptable tolerances. The grader matches those pins to the submitted PCB, follows the realized copper, and runs deterministic checks or ngspice simulation [ngspice contributors, 2026]. A different internal circuit can score well if it produces the required boundary behavior. A visually plausible board can fail if a connector pin is isolated, swapped, shorted, or not driven by the needed circuitry.

Contributions. This paper makes five contributions.

- We introduce EDA Bench, a public benchmark of 35 source-backed KiCad PCB reconstruction tasks spanning connector breakouts, power-path circuits, controller boards, compute adapters, motor drivers, and high-density carrier boards.
- We define a reproducible harness protocol in which agents receive only task-visible runtime packs and must produce complete KiCad projects, while references, source snapshots, expected-I/O files, test cases, and grader internals remain in the grading pack.
- We develop an execution-based PCB grader that checks external pins, routed copper, ngspice behavior, task-family requirements, active circuitry, KiCad ERC/DRC diagnostics, and gross manufacturability.
- We validate the grader with frozen references, structural failures, realistic single-fault edits, historical upstream versions, and representation changes, reporting both detected faults and grader blind spots.
- We report public-provenance baselines for six web-enabled harness configurations, including final projects, grader outputs, usage summaries, cost estimates, and regrade commands, and show that current evaluated frontier-model harnesses remain far from functional PCB reconstruction.

2 Benchmark overview

EDA Bench evaluates PCB reconstruction as an execution task. For each task, a harness receives a fixed prompt, runtime task pack, tool environment, model configuration, and access policy, then must produce a complete KiCad project in `/workspace/final_project`. The grader parses the files, finds external connectors and routed copper, applies the task’s test signals and measurements, runs simulation where applicable, checks ERC and DRC diagnostics, and returns a normalized score.

Table 1 summarizes the pipeline. Each released task comes from a public hardware project and has a frozen reference design, but the evaluated runtime does not mount the reference, source snapshot, test cases, expected-I/O files, or grader implementation. This keeps the answer out of the container while leaving the released artifact inspectable. Because the sources are public and the main baselines are web-enabled, EDA Bench measures reconstruction under a declared public-source access policy rather than closed-book invention.

Scores should be interpreted as properties of complete evaluated harnesses, not isolated model weights. A harness may include prompting, web search, source discovery, KiCad tool use, repair loops, and project-generation logic. This systems-level framing is intentional because practical EDA automation depends on whether an agent can operate the full workflow, not only on whether a model can emit plausible schematic or layout text in one shot.

The score concerns external electrical behavior rather than visual similarity. A board can use a different internal implementation if it exposes the required signals at the required pins. It can fail despite looking plausible if nets are missing, isolated, shorted, swapped, unrouted, or not driven by active circuitry. The score is not a fabrication certificate, regulatory review, thermal analysis, EMC validation, or vendor-accurate silicon simulation.

Table 1: EDA Bench evaluation pipeline. Each submission is evaluated as a complete harness output rather than as text, a screenshot, or a component list.

Stage	Artifact or operation
Task input	The harness receives a fixed prompt, runtime task pack, tool environment, model configuration, and access policy.
Harness execution	The agent operates in the Dockerized KiCad environment and writes a project to <code>/workspace/final_project</code> .
Project parsing	The grader locates KiCad files and extracts connectors, pads, nets, board outline, and routed copper geometry.
Port matching	Submitted connector pins are matched to the external pins and measurements listed by the task.
Execution checks	The grader applies KiCad ERC and DRC diagnostics, ngspice simulations, routed-I/O checks, parasitic models, and short-fault checks.
Score reporting	The grader returns a normalized task score, failure diagnostics, build status, and provenance artifacts.

Table 2: Task distribution and representative categories in EDA Bench.

Difficulty	Tasks	Representative task families
Very easy	2	Compact connector breakouts and small adapters
Easy	5	I/O breakouts, logic-level conversion, USB/UART basics
Medium	4	Power-path boards, serial transceivers, small controller boards
Hard	12	Compute adapters, sensor/camera boards, multi-connector carriers
Very hard	7	Dense mixed-signal boards and larger interface carriers
Extreme	5	Large baseboards and high-density compute boards

3 Benchmark tasks

EDA Bench contains 35 released source-backed tasks. Each task is anchored to a public upstream hardware project and a frozen reference design. The public task set covers six difficulty levels, shown in Table 2. Five additional candidate tasks were quarantined because their redistribution terms require permission or an acquisition-script replacement.

The task set was deliberately constructed to represent the major skills and project types in public, reproducible PCB reconstruction. Across six difficulty levels, it covers compact breakouts, power paths, controllers, camera and compute adapters, motor drivers, and dense carrier boards. A task is included only when it has (i) a redistributable public source or reproducible acquisition path, (ii) a complete KiCad board that can be frozen as a reference, (iii) identifiable external connectors, (iv) deterministic grader checks, and (v) a task family not already over-represented. This coverage-driven selection gives every task strong ground truth and avoids spending the evaluation budget on near-duplicates. Its external-validity boundary is public open-hardware designs with inspectable connector behavior: proprietary industry projects and analog-performance, RF, thermal, power-sequencing, and production-sign-off workflows are not yet covered.

The task catalog includes compact breakouts and converters such as `usb_c_female_breakout`, `mcp23017_breakout`, and `rs485_transceiver_breakout`. It also includes power and control boards such as `bq24295_power_path_board`, `picopd`, and `robotont_driver_board`, compute and camera or display adapters such as `cm4_csi_adapter`, `m2_pcie_adapter`, and `ov9281_dual_camera_board`, and large carrier boards such as `cm4_baseboard`, `jetson_orin_baseboard`, and `zynq_som`.

Each task has two artifact forms. The runtime task pack is mounted for the evaluated agent and contains the prompt, task metadata, and runtime assets. It does not include the reference project, source snapshot, test cases, expected-I/O files, or grader implementation. The full grading pack is public and adds the reference, upstream snapshot, controlled-fault submissions, license notes, `io_contract.json`, `functional_contract.json`, and local tests. The filenames remain for compatibility, but in plain language these two JSON files say which external pins exist, which signals to apply, what to measure, and which task-specific checks to run.

116 4 Harness protocol

117 A harness is evaluated as a complete system. It receives the task prompt and must produce a KiCad
118 project in `/workspace/final_project`. Built-in harnesses run inside Docker with KiCad, ngspice,
119 Python, Node, the declared web policy, and the configured model interface. The evaluated systems
120 are Codex CLI with GPT-5.5 web search at four reasoning settings, plus Pi Coding Agent—an open-
121 source terminal agent that can edit files and run shell tools [Zechner and contributors, 2026]—with
122 Gemini 3.1 Pro Preview or DeepSeek V4 Pro.

123 This protocol evaluates system architecture, not only a model weight. A one-shot generator, a
124 tool-using agent, and a multi-agent harness can all target the same task interface. This distinction
125 matters because EDA work is iterative: the model may need to create files, run KiCad tools, inspect
126 failures, search for upstream sources, and repair invalid output.

127 Each run uploads provenance to Hugging Face. Reports include prompts, command lines, usage
128 summaries, grader outputs, final projects, and generated artifacts. Large Docker image uploads and
129 raw model transcripts can be disabled to minimize storage usage, but reports still retain byte-count
130 summaries for omitted transcript content.

131 5 Scoring method

132 The grader asks a concrete question: if another device were connected to the named board pins, would
133 the submitted PCB expose the required paths and responses? Missing or malformed projects receive
134 zero. For a parseable project, the grader runs seven steps: locate the KiCad files; run ERC and DRC;
135 extract connectors, pads, nets, and routed copper; match submitted pins to the task’s external pins;
136 apply the listed test signals and measurements; compute family-specific checks and failure caps; and
137 return a score with diagnostics.

138 The primary score is a normalized task score in $[0, 1]$. A benchmark aggregate is a per-task difficulty-
139 weighted mean:

$$S = \frac{\sum_{i=1}^N w(d_i) s_i}{\sum_{i=1}^N w(d_i)}, \quad w(d_i) \in \{1.0, 1.5, 2.0, 3.0, 4.0, 5.0\}. \quad (1)$$

140 We also report the unweighted mean over tasks. Build success means that the submission produced a
141 KiCad project that the grader could parse and evaluate. It is not equivalent to functional success.

142 The main score comes from three observable properties: whether connector pins mean the right
143 thing, whether routed copper realizes the required paths, and whether the needed active circuitry and
144 gross physical layout are plausible. Deterministic ngspice signals are applied at declared external
145 pins where the task supports simulation. Missing copper, swapped pins, shorts, absent or unpowered
146 active devices, missing high-speed pair families, and gross fabrication problems cap the score. Exact
147 reference designators, component inventory, footprint similarity, outline similarity, and route-shape
148 similarity are diagnostics rather than primary score signals.

149 The measurements produce partial scores for pin mapping, routed continuity, simulated response, and
150 task-family checks. Severe structural failures then cap the combined score. This allows partial credit
151 for partially correct behavior while limiting credit for a board that passes unrelated checks but has a
152 short, an isolated connector, swapped pins, or no active circuit.

153 6 Artifact validation

154 Because EDA Bench uses task-specific checks, the release includes controlled boards that calibrate
155 the score. The positive control is the frozen reference and must score 1.0. The severe negative control
156 removes active internal circuitry when available; for passive boards it removes an external boundary.
157 It must score below the release threshold of 0.75.

158 **What validation establishes.** The grader is not a vendor-accurate electrical simulator. Its intended
159 validity criterion is narrower: score changes should track changes to the external behavior named
160 by the task. Positive controls test that real human-designed boards score well. Fault controls test

Table 3: Representative task-family checks layered on the shared external-I/O grader. These are deterministic benchmark checks, not vendor-accurate silicon or channel models.

Task family	Declared behavior	Common hard caps
Connector breakouts and simple adapters	Connector pins and power/ground pads, continuity, shorts, routed coverage, pin/net semantic agreement	Swapped pins, split nets, isolated pads, power-ground shorts
Serial and level-shifting interfaces	USB/UART/RS-485/logic-side boundary ports, signal transfer families, active-device realization	Missing active path, weak external mapping, missing routed I/O
Power-path and charger boards	Input, battery/load, configuration, and status ports, power path presence, external safety	Missing active power role, shorts, missing component-function profile
Camera/display/compute carriers	High-density connector families and differential pairs, routed families, pair plausibility	Missing high-speed pair families, weak routing, shorts, split nets
Large mixed-signal baseboards	Multi-rail power, GPIO, debug, serial, high-speed, and peripheral interfaces	Incomplete connector semantics, missing active realization, shorts

Table 4: Grader-validation evidence. These controls calibrate a boundary-behavior score, not complete manufacturing or vendor-silicon review.

Evidence	What it supports
35/35 released frozen human-designed reference PCBs score 1.0	The tasks are solvable, and the grader accepts the selected upstream designs.
35/35 released structural fail canaries score at most 0.15	The score is not reducible to file existence, KiCad syntax, or component-shell matching.
162 candidate-set mutation controls pass validation	The grader is sensitive to shorts, unrouted nets, isolated pads, missing outlines, and missing high-speed-like routing.
Renaming internal reference designators scores 1.0 on all 35 references	The broad fault-sweep score does not depend on internal component names.
Renaming the two USB-C connector references scores 0.992	External connector names have a small, unintended effect.
Changing plausible USB-C route widths to 0.35 mm scores 0.25	The current grader is too sensitive to a benign routing representation change.
Replacing six USB-C signal-net names with generic names scores 0.65	The current grader partly relies on semantic net names even when topology is unchanged.
Tiny-trace USB-C control scores at most 0.35	Manufacturability caps affect scores for physically implausible routing.

161 changes that break routing, power, pin mapping, or active circuitry. Representation controls test
162 whether benign edits change the score; as Table 4 shows, several do.

163 A full local sweep on May 6, 2026 verified all 35 released task canaries: every pass canary scored
164 1.0, and every fail canary scored at most 0.15. The generated-mutation sweep graded 162 mutation
165 canaries across the original 40-task candidate set. All tasks passed, with every reference scoring
166 1.0 and every structural fail canary scoring at most 0.15. Mutations covered missing board outlines,
167 unrouted external nets, isolated external pads, power or signal pads tied to ground, and missing
168 high-speed-like routes for high-speed carrier tasks.

169 These controls do not show that the grader replaces expert board review or captures vendor-specific
170 silicon behavior. They show that it accepts the selected reference boards and rejects several common
171 function-breaking edits. They also expose representation sensitivity that the original validation
172 discussion understated: internal component renaming is harmless, but connector names, signal-net
173 names, and even a plausible uniform route-width change can reduce the score. This is a grader
174 limitation, not an electrical failure of those boards.

Table 5: Single-fault sweep over the 35 frozen references. “Detected” counts scores at or below 0.75; applicability differs when a board lacks the edited feature.

Board variant	Applicable	Mean score	Detected
Renamed internal reference designators	35	1.000	0
Two external connector pins swapped	35	0.672	19
One external pad disconnected	35	0.892	6
One external route removed	34	0.766	12
One signal tied to ground	35	0.760	15
Active device loses all power or ground	31	0.698	18
One passive component removed	34	0.987	1
One active device removed	32	0.797	9
One high-speed route removed	28	0.811	8

Table 6: Complete web-enabled harness baselines. The primary score is difficulty-weighted, and the unweighted score is the mean task score.

Harness	Weighted	Unweighted	Builds
Codex GPT-5.5 web low	0.0124	0.0143	31/35
Codex GPT-5.5 web medium	0.0279	0.0314	35/35
Codex GPT-5.5 web high	0.0293	0.0229	11/35
Codex GPT-5.5 web xhigh	0.0458	0.0500	21/35
Pi Gemini 3.1 Pro Preview web high	0.0113	0.0086	1/35
Pi DeepSeek V4 Pro web high	0.0000	0.0000	0/35

Realistic single-fault sweep. We applied ten additional, deterministic edits across all applicable references. Table 5 reports every fault family rather than only the successful controls. The grader detects many pin swaps, missing routes, shorts, and unpowered active circuits, but it often misses a single disconnected pad, passive removal, active-device removal, or high-speed route removal. For example, 16 of 35 pin-swapped boards still score at least 0.95, and 23 of 32 boards with one active device removed still score at least 0.95. These are important false negatives. The score is useful for the failures it checks, but it is not a general certificate that every component or route is correct.

7 Experiments

We evaluate six web-enabled harness configurations on the 35 released tasks. All runs use the same task prompts, task packs, Docker image, and grading protocol. Table 6 reports a May 6, 2026 no-model regrade of saved final projects under the submitted grader revision.

No evaluated harness achieves a high absolute score. The main empirical finding is that build success substantially overestimates functional PCB reconstruction. The medium reasoning-level Codex harness builds all 35 released projects but scores only 2.79%. The xhigh harness builds fewer projects than medium but scores higher. These results indicate that executable KiCad output is not a sufficient metric for EDA agents and that reasoning and tool-use choices affect the distribution of functional failures.

Difficulty breakdowns further indicate that the benchmark is not saturated by easy tasks. The xhigh harness reaches 13.75% on medium tasks and 7.08% on hard tasks, but only 2.00% on extreme tasks. The gap grows as boards expose more required pins, active circuitry, dense routing, or high-speed interfaces.

Statistical interpretation. The result table reports one saved project per task and configuration, not repeated independent model runs. We therefore do not claim statistically significant pairwise model rankings. As a task-composition sensitivity check, we resample the 35 tasks with replacement. The xhigh row has a task-bootstrap 95% interval of [0.0282, 0.0662] and ranks first in 79.1% of resamples; the medium and high rows have intervals [0.0169, 0.0411] and [0.0085, 0.0587]. The same xhigh row remains first under the paper’s difficulty weights (0.0458), an unweighted task mean (0.0500), and equal weighting of the six tiers (0.0475). These intervals measure sensitivity to which tasks are included; they are not stochastic-run confidence intervals.

7.1 Access-policy calibration

EDA Bench records web access as part of the system definition, but the archived no-web attempts do not support a web-versus-no-web comparison. Manual provenance review found that all six no-web campaigns used invalidated authentication tokens and failed before producing usable model outputs. They are infrastructure failures and are removed from the capability analysis. A valid access-policy ablation would require fresh authenticated repeats with identical model, prompt, runtime, and task pack. We do not have that experiment.

Public-source reuse audit. Because the main harnesses can browse public upstream repositories, we compared every parseable saved board with its frozen reference. Across the four Codex configurations there are zero byte-identical PCB files and zero matches after canonicalizing reference-designator names and ordering. Median topology similarity ranges from 0.216 to 0.520 and median route-geometry similarity is 0.000. The only near-reference route case is the sole parseable Pi/Gemini board (route Jaccard 0.825, topology 1.000). This audit can detect direct artifact reuse but cannot rule out manual transcription or use of documentation, and the archived provenance omits full browsing transcripts. We therefore retain the public-source reconstruction framing and make no claim about URL-by-URL source use.

8 Failure analysis

The results support three benchmark-level conclusions. First, current evaluated frontier-model harnesses are far from solving functional PCB reconstruction: the best evaluated configuration scores only 4.58% overall on the released task set. Second, syntactic validity is not sufficient: a harness can produce buildable KiCad projects for every released task and still fail most functional checks. Third, harness architecture matters: Codex GPT-5.5 configurations with different reasoning settings produce different build and score tradeoffs, so EDA Bench evaluates complete systems rather than base models alone.

Saved grading artifacts expose the main failure modes. The archived pre-override grader outputs contain non-exclusive diagnostic events, so one board can contribute several counts. For the all-building medium run, we count 73 electrical-short events, 50 missing or incorrectly powered functional-circuit events, 49 routing/continuity failures, 35 wrong external-interface mappings, four high-speed-geometry failures, and four simulation/transfer failures. For xhigh, the corresponding counts are 20, 22, 24, 18, five, and zero across 19 parseable archived boards. These counts describe grader diagnoses rather than independent board-level labels, but they make the qualitative failure analysis reproducible.

These caps correspond to concrete functional failures rather than parser failures. Submissions often instantiate connector footprints with plausible labels, but leave required I/O pads isolated, split required nets across disconnected copper, swap external pin semantics, omit active circuitry, or route high-speed-like interfaces implausibly. These errors explain why KiCad parseability and component presence are weak proxies for functional PCB reconstruction.

9 Related work

Execution-based software-agent benchmarks. Execution-based software benchmarks evaluate agents by running the artifacts they produce. SWE-bench frames software engineering as repository-level issue resolution, where an agent edits a real codebase and must pass tests associated with the target issue [Jimenez et al., 2023]. OSWorld evaluates multimodal agents in real desktop and web environments with custom execution-based evaluation scripts [Xie et al., 2024]. ProgramBench studies a more reconstructive setting in which agents rebuild software from an executable and documentation, and behavioral tests are generated to compare the candidate program with the reference executable [Yang et al., 2026]. EDA Bench adopts this behavioral-evaluation perspective but applies it to PCB reconstruction, where correctness depends on realized geometry, external connectivity, and electrical-interface behavior rather than software tests alone.

CAD generation and geometric evaluation. Text2CAD introduced a large-scale text-to-parametric-CAD dataset and model for generating CAD command sequences from natural-language prompts

[Khan et al., 2024]. CadEval and CAD Arena provide evaluation settings for text-to-CAD models using geometric validity, rendering, and prompt-level comparisons [Epoch AI, 2026, CAD Arena, 2026]. CADSmith further emphasizes programmatic geometric validation for generated CadQuery models [Barkley et al., 2026]. These works motivate execution-based evaluation for engineering artifacts, but their primary correctness signals are geometric rather than electrical. PCB reconstruction requires connectivity, routing, short avoidance, active-circuit realization, and external-I/O behavior in addition to shape or layout plausibility.

Circuit, schematic, and PCB benchmarks. Recent EDA benchmarks study several adjacent tasks. CircuitLM and PCBSchemaGen generate or synthesize circuit schematics from natural-language prompts and use structural or constraint-based checks to improve schematic validity [Al Hasan et al., 2026, Zou et al., 2026]. SchGen studies PCB schematic generation using semantic-grounded code representations [Luo et al., 2026]. OmniSch evaluates multimodal schematic understanding and schematic-to-graph reasoning over real-world diagrams [Lu et al., 2026]. HWE-Bench evaluates board-level schematic design with datasheet context and hardware bug repair over repository-scale hardware projects [Qiu et al., 2026, Cui et al., 2026]. PCB-Bench evaluates LLM reasoning over placement, routing, and PCB design comprehension using text, images, and complete PCB projects [Li et al., 2026]. EDA Bench differs by requiring an agent to produce a complete routed KiCad PCB project and by scoring the realized board through external-I/O simulation and PCB-geometry checks rather than schematic-only validity, visual reasoning, or placement/routing question answering.

10 Limitations

EDA Bench is not a hidden-answer or closed-book benchmark. The released tasks are public-source, and upstream designs may be discovered through web search. This determines the supported claim: scores measure EDA task solving under fixed prompts, tools, task-pack revisions, provenance, and access policy, not closed-book invention.

The 35-task design gives EDA Bench broad skill coverage while keeping a complete, tool-using sweep practical. This is the same scale regime used by established expensive open-ended benchmarks: MLAgentBench uses 13 experimentation tasks, RE-Bench uses seven research-engineering environments, PaperBench uses 20 full-paper replications, and PostTrainBench applies one open-ended post-training task formulation across four base models and seven target benchmarks [Huang et al., 2024, Wijk et al., 2024, Starace et al., 2025, Rank et al., 2026]. Curated small subsets can also preserve useful signals from larger suites. TinyBenchmarks reports that 100 selected examples can estimate the 14,000-example MMLU benchmark, while SWE-bench Verified Mini reports that 50 of 500 tasks approximately preserve performance, test-pass-rate, and difficulty distributions across 16 models while reducing storage from 130 GB to 5 GB [Polo et al., 2024, Hobbhahn, 2025].

Within its stated population of public, source-backed KiCad projects with externally testable behavior, EDA Bench deliberately covers different board families, functional skills, and difficulty levels. Each task contributes a real source design, frozen reference, expected-I/O files, controlled faults, license review, and grader support. The resulting size makes complete six-harness sweeps, an all-task fault audit, alternative weighting, and task-bootstrap analysis practical. The reported claim is therefore well matched to the evidence: current evaluated systems often produce parseable boards but rarely reproduce the required external electrical behavior on this diverse suite. Additional source-backed packs and a sealed private split can extend coverage to proprietary and currently omitted workflows.

The grader is not a substitute for manufacturing review. It uses deterministic generic behavioral models and does not prove electrical safety, regulatory compliance, high-speed protocol compliance, thermal reliability, power sequencing, or production readiness. Richer per-task SPICE, IBIS, S-parameter, and field-solver models would improve fidelity. The single-fault sweep shows concrete false negatives, including missed passive, active-device, and route removals. The USB-C representation controls also show false rejections: a plausible uniform route-width edit scores 0.25, and generic signal-net names score 0.65 despite unchanged topology.

We have not run independent stochastic repeats of the model harnesses, and the task-bootstrap intervals do not replace them. We also have not measured agreement between the grader and blinded PCB engineers. The reference, fault, and historical-version studies test controlled artifact changes, not human correlation. Finally, the storage-light provenance omits complete model/tool transcripts,

so this version supports artifact-level failure analysis but not a step-by-step study of agent reasoning or browsing. These are missing experiments, not claims deferred by wording.

Task artifacts inherit mixed upstream hardware licenses. Five candidate tasks with unlicensed, upstream-terms-only, noncommercial, or otherwise restricted source snapshots are quarantined from the public release until they receive explicit redistribution permission or acquisition-script replacements.

11 Conclusion

EDA Bench evaluates whether language-model agents can reconstruct PCB designs that behave correctly at their external pins. It is built from public hardware projects, scored by routed-copper and I/O checks, and evaluated as a full system rather than a single text response. Current systems often produce parseable KiCad artifacts but rarely satisfy those external checks. The grader catches important failures and also has documented blind spots; both are part of the benchmark result.

References

- Khandakar Shakib Al Hasan, Syed Rifat Raiyan, Hasin Mahtab Alvee, and Wahid Sadik. CircuitLM: A Multi-Agent LLM-Aided Design Framework for Generating Circuit Schematics from Natural Language Prompts. arXiv:2601.04505, 2026. URL: <https://arxiv.org/abs/2601.04505>.
- Jesse Barkley, Rumi Loghmani, and Amir Barati Farimani. CADSmith: Multi-Agent CAD Generation with Programmatic Geometric Validation. arXiv:2603.26512, 2026. URL: <https://arxiv.org/abs/2603.26512>.
- CAD Arena. CAD Arena: Open Benchmark for AI-Generated Parametric CAD. 2026. URL: <https://cadarena.dev/>.
- Fan Cui, Hongyuan Hou, Zizhang Luo, Chenyun Yin, and Yun Liang. HWE-Bench: Benchmarking LLM Agents on Real-World Hardware Bug Repair Tasks. arXiv:2604.14709, 2026. URL: <https://arxiv.org/abs/2604.14709>.
- Epoch AI. CadEval. 2026. URL: <https://epoch.ai/benchmarks/cad-eval>.
- Marius Hobbhahn. SWE-bench Verified Mini. GitHub repository, 2025. URL: <https://github.com/mariushobbhahn/SWEBench-verified-mini>.
- Qian Huang, Jian Vora, Percy Liang, and Jure Leskovec. MAgentBench: Evaluating Language Agents on Machine Learning Experimentation. In *Proceedings of the 41st International Conference on Machine Learning*, 2024. URL: <https://arxiv.org/abs/2310.03302>.
- Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? arXiv:2310.06770, 2023. URL: <https://arxiv.org/abs/2310.06770>.
- Mohammad Sadil Khan, Sankalp Sinha, Talha Uddin Sheikh, Didier Stricker, Sk Aziz Ali, and Muhammad Zeshan Afzal. Text2CAD: Generating Sequential CAD Models from Beginner-to-Expert Level Text Prompts. In *Advances in Neural Information Processing Systems*, volume 37, 2024. URL: <https://arxiv.org/abs/2409.17106>.
- KiCad Project. KiCad Documentation. 2026. URL: <https://docs.kicad.org/>.
- Jindong Li, Lianrong Chen, Bin Yang, Jiadong Zhu, Ying Wang, Yuzhe Ma, and Menglin Yang. PCB-Bench: Benchmarking LLMs for Printed Circuit Board Placement and Routing. In *International Conference on Learning Representations*, 2026. URL: <https://openreview.net/forum?id=Q5QLu7XTWx>.
- Taiting Lu, Kaiyuan Lin, Yuxin Tian, Yubo Wang, Muchuan Wang, Sharique Khatri, Akshit Kartik, Yixi Wang, Amey Santosh Rane, Yida Wang, Yifan Yang, Yi-Chao Chen, Yincheng Jin, and Mahanth Gowda. OmniSch: A Multimodal PCB Schematic Benchmark For Structured Diagram Visual Reasoning. arXiv:2604.00270, 2026. URL: <https://arxiv.org/abs/2604.00270>.

353 Qinpei Luo, Ruichun Ma, Xinyu Zhang, and Lili Qiu. SchGen: PCB Schematic Generation
 354 with Semantic-Grounded Code Representations. OpenReview preprint, 2026. URL: <https://openreview.net/forum?id=TyWs6rWWhb>.
 355

356 ngspice contributors. ngspice, the open-source SPICE circuit simulator. 2026. URL: <https://ngspice.sourceforge.io/>.
 357

358 Weibo Qiu, Yin hao Xiao, and Runyu Pan. HWE-Bench: Can Language Models Perform Board-level
 359 Schematic Designs? arXiv:2603.18102, 2026. URL: <https://arxiv.org/abs/2603.18102>.

360 Felipe Maia Polo, Lucas Weber, Leshem Choshen, Yuekai Sun, Gongjun Xu, and Mikhail Yurochkin.
 361 tinyBenchmarks: evaluating LLMs with fewer examples. In *Proceedings of the 41st International*
 362 *Conference on Machine Learning*, 2024. URL: <https://arxiv.org/abs/2402.14992>.

363 Ben Rank, Hardik Bhatnagar, Ameya Prabhu, Shira Eisenberg, Karina Nguyen, Matthias Bethge, and
 364 Maksym Andriushchenko. PostTrainBench: Can LLM Agents Automate LLM Post-Training?
 365 arXiv:2603.08640, 2026. URL: <https://arxiv.org/abs/2603.08640>.

366 Giulio Starace, Oliver Jaffe, Dane Sherburn, James Aung, Jun Shern Chan, Leon Maksin, Rachel Dias,
 367 Evan Mays, Benjamin Kinsella, Wyatt Thompson, Johannes Heidecke, Amelia Glaese, and Tejal
 368 Patwardhan. PaperBench: Evaluating AI’s Ability to Replicate AI Research. arXiv:2504.01848,
 369 2025. URL: <https://arxiv.org/abs/2504.01848>.

370 Hjalmar Wijk, Tao Lin, Joel Becker, Sami Jawhar, Neev Parikh, Thomas Broadley, Lawrence Chan,
 371 Michael Chen, Josh Clymer, Jai Dhyani, Elena Ericheva, Katharyn Garcia, Brian Goodrich,
 372 Nikola Jurkovic, Megan Kinniment, Aron Lajko, Seraphina Nix, Lucas Sato, William Saunders,
 373 Maksym Taran, Ben West, and Elizabeth Barnes. RE-Bench: Evaluating frontier AI R&D
 374 capabilities of language model agents against human experts. arXiv:2411.15114, 2024. URL:
 375 <https://arxiv.org/abs/2411.15114>.

376 Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing
 377 Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio
 378 Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking Multimodal
 379 Agents for Open-Ended Tasks in Real Computer Environments. arXiv:2404.07972, 2024. URL:
 380 <https://arxiv.org/abs/2404.07972>.

381 Mario Zechner and contributors. Pi Coding Agent. Open-source terminal coding-agent harness, 2026.
 382 URL: <https://github.com/badlogic/pi-mono/tree/main/packages/coding-agent>.

383 John Yang, Kilian Lieret, Jeffrey Ma, Parth Thakkar, Dmitrii Pedchenko, Sten Sootla, Emily McMilin,
 384 Pengcheng Yin, Rui Hou, Gabriel Synnaeve, Diyi Yang, and Ofir Press. ProgramBench: Can
 385 Language Models Rebuild Programs From Scratch? arXiv:2605.03546, 2026. URL: <https://arxiv.org/abs/2605.03546>.
 386

387 Huanghaohe Zou, Peng Han, Emad Nazerian, and Alex Q. Huang. PCBSchemaGen: Constraint-
 388 Guided Schematic Design via LLM for Printed Circuit Boards. arXiv:2602.00510, 2026. URL:
 389 <https://arxiv.org/abs/2602.00510>.

390 A Artifact availability and provenance

391 The source repository contains the runner and harness code, task-pack loader and grader support, pub-
 392 lic documentation, package tests, and the task-pack manifest. Python wheels and source distributions
 393 exclude concrete task instances, reference projects, upstream snapshots, and canary submissions. The
 394 Hugging Face task dataset contains runtime task packs, full grader task packs, prompts, references,
 395 upstream snapshots, canary submissions, task metadata, Croissant metadata, and license/provenance
 396 notes.

397 The current public task dataset is hosted at <https://huggingface.co/datasets/eda-bench-neurips-2026/eda-bench-tasks>. The paper results use task-pack revision
 398 c64b9169d6d8c0d73614e14c7050cf036aeb1559. The public provenance dataset is hosted at
 399 <https://huggingface.co/datasets/eda-bench-neurips-2026/eda-bench-provenance>.
 400 The public code release is hosted at <https://github.com/CAD-bench/eda-bench>.
 401

Run provenance includes task prompts, command lines, usage summaries, grader outputs, final projects, and generated artifacts. Storage-light campaigns can omit full Docker image uploads and raw model transcripts, but retain final reports, final projects, grading artifacts, and byte-count summaries for omitted transcript content.

B Validation controls

Table 7: Reviewer-facing validation controls for score interpretation.

Control	Observed result
Frozen reference project	35/35 released references score 1.0
Structural fail canary	35/35 released fail canaries score at most 0.15, below the 0.75 release threshold
Generated mutation canaries	162 generated mutation canaries across the original 40-task candidate set pass validation
Calibrated task-family checks	USB-C, RS-485, BQ24295, and M.2/PCIe references score 1.0 and their mutation canaries remain below threshold
Internal refdes rename sweep	All 35 references remain at 1.0
External connector refdes rename	USB-C reference scores 0.992 rather than 1.0
Missing routed copper	USB-C and NanoUPDI route-removal controls score 0.0
Plausible route-width edit	USB-C route-width edit scores 0.25, an adverse representation-sensitivity result
Generic signal-net names	USB-C topology with six generic signal names scores 0.65
Implausibly tiny traces	Manufacturability cap applies and USB-C tiny-trace control scores at most 0.35
Missing or unparsable project	Returns 0.0 and build failure

The broader single-fault sweep in Table 5 applies swapped-pin and active-power edits across every applicable released task. Unlike the older pass/fail threshold controls above, it keeps and reports high-scoring false negatives. The new USB-C representation controls likewise retain the observed low scores rather than treating the intended invariance as established.

B.1 Historical upstream versions

As a second check, we graded 13 distinct board directories preserved in exact historical commits or versioned releases of seven pinned upstream projects. These are not synthetic mutations and are byte-distinct from the frozen task reference except where noted. Scores range from 0.050 to 1.000: five versions score 1.000, one scores 0.997, one scores 0.550, one scores 0.050, and five score 0.200. For example, two older USB-C breakout releases score 1.000, while a camera-adapter version immediately preceding an upstream I2C-line swap scores 0.200. Low scores can reflect a real product change rather than grader error, so this check supports sensitivity to released design changes but does not replace independently authored equivalence or engineer correlation.

Table 8: Historical-version examples from pinned upstream repositories.

Task	Historical version	Score
CM4 CSI adapter	immediately before upstream I2C-line swap	0.200
FTDI toolkit	before final DRC/ERC fixes	0.997
FTDI toolkit	earlier layout revision	0.050
NanoUPDI	v2.2 layout before fabrication cleanup	1.000
NanoUPDI	earlier v2 layout	0.550
OV9281 dual camera	immediately before lens-DNP fix	1.000
Pi485	v1.1.1 standalone board	0.200
Pi485	earlier high-side-switching board	0.200
USB-C breakout	v1.0.0 / v1.1.1	1.000 / 1.000
USB-UART CH340C	v1.0.0 / v1.1.1	0.200 / 1.000

Table 9: Common functional failure modes exposed by the grader. These failures are typical of buildable submissions and illustrate why KiCad parseability is not sufficient for functional PCB reconstruction.

Failure mode	Observable symptom	Why it affects score
Split required external nets	Pins that should belong to the same external interface are assigned to disconnected copper regions or separate nets	Boundary stimuli cannot propagate through the declared interface, even when connector footprints are present
Isolated external I/O pads	Required pads are placed but not connected to routed copper or active circuitry	The board exposes a physical pad without realizing the electrical behavior required by the task
Wrong pin or net mapping	External connector pins are swapped, mislabeled, or attached to semantically incorrect nets	The board may appear visually plausible while driving or observing the wrong external signal
Missing active realization	Components required for level shifting, power-path behavior, transceiver behavior, or control logic are absent or not connected	Passive connectivity alone cannot implement the task-required circuit behavior
Weak active-device power integrity	Active devices are present but lack required power, ground, or enable connectivity	The simulated or inferred behavior cannot be attributed to a realizable powered circuit
Implausible high-speed geometry	Differential or high-speed-like interface families are missing, weakly routed, or routed with implausible geometry	Dense carrier and compute tasks require not only pin labels but routed interface families with plausible physical realization

421 **D Baseline provenance**

422 The May 6, 2026 result table regrades saved final projects without rerunning model har-
 423 nesses. Source runs were: `codex__gpt-5.5-web-low_20260505T200208Z_2719115`, `codex__`
 424 `_gpt-5.5-web-medium_20260506T010715Z_2812175`, `codex__gpt-5.5-web-medium_`
 425 `20260506T010715Z_2812182`, `codex__gpt-5.5-web-medium_20260506T032737Z_2884928`,
 426 `codex__gpt-5.5-web-medium_20260506T010716Z_2812190`, `codex__gpt-5.5-web-high_`
 427 `20260505T200208Z_2719113`, `codex__gpt-5.5-web-xhigh_20260506T032737Z_2884940`,
 428 `codex__gpt-5.5-web-xhigh_20260506T032737Z_2884938`, `codex__gpt-5.5-web-xhigh_`
 429 `20260506T032737Z_2884943`, `codex__gpt-5.5-web-xhigh_20260506T034126Z_2891808`,
 430 `pi__gemini-3.1-pro-preview-web-high_20260505T184850Z_2697190`, and
 431 `pi__deepseek__deepseek-v4-pro-web-high_20260505T200208Z_2719116`.

432 The executable regrade entry point is `uv run regrade-provenance`. For example, the Codex
 433 medium row is reproduced by passing its four source runs:

```
434 uv run regrade-provenance \  

435 --run codex__gpt-5.5-web-medium_20260506T010715Z_2812175 \  

436 --run codex__gpt-5.5-web-medium_20260506T010715Z_2812182 \  

437 --run codex__gpt-5.5-web-medium_20260506T032737Z_2884928 \  

438 --run codex__gpt-5.5-web-medium_20260506T010716Z_2812190
```

439 By default the command uses the current released task manifest, downloads saved
 440 submission/final_project directories from Hugging Face provenance, and runs the current
 441 local grader.

442 Completed no-web calibration runs are present under these provenance paths:
 443 `evals/harnesses/codex__gpt-5.5-no_web-medium_20260507T033319Z_3432012`,
 444 `evals/harnesses/codex__gpt-5.5-no_web-high_20260507T033319Z_3432018`,
 445 `evals/harnesses/codex__gpt-5.5-no_web-xhigh_20260507T033319Z_3432017`,
 446 `evals/harnesses/codex__gpt-5.4-no_web-xhigh_20260507T033528Z_3439753`, `evals/`

harnesses/codex__gpt-5.4-mini-no_web-medium_20260507T033724Z_3448333, and
evals/harnesses/codex__gpt-5.4-mini-no_web-high_20260507T033724Z_3448341.

E Cost and compute accounting

The completed GPT-5.5 web runs used storage-light provenance. The combined Codex GPT-5.5 low/medium/high/xhigh campaign uploaded about 185.6 MiB of Hugging Face provenance. Across all six final harnesses, the total uploaded provenance was about 193.1 MiB.

Table 10: Recorded Codex GPT-5.5 web campaign accounting.

Harness	Estimated OpenAI API USD	Estimated Codex credits	HF provenance
Codex low	38.22	955.41	48.9 MB
Codex medium	84.40	2110.05	49.5 MB
Codex high	49.39	1234.64	27.6 MB
Codex xhigh	182.75	4568.64	68.6 MB

The estimates are computed from recorded token usage, current provider pricing, and the Codex token-based rate card. They are included to make leaderboard submissions auditable and to discourage hidden cost and provenance blowups.

F Release and license gate

EDA Bench separates runtime prompts from full task packs with references, snapshots, controlled faults, and expected-I/O files. For NeurIPS release, full packs with unlicensed, upstream-terms-only, noncommercial, or otherwise restricted source snapshots must be handled by one of three policies: obtain explicit redistribution permission, remove the source snapshot and provide a reproducible acquisition script, or quarantine the task from the public full pack while retaining runtime metadata.

The current public release quarantines five candidate tasks under this conservative rule: cm5io_official, mixed_signal_stm32_dev_board, rp2350b_dev_board, stm32f_audio_codec, and ef28_badge. The public score-supported full-pack release has 35 tasks. These task ids may remain documented as candidates, but their references, source snapshots, and canaries are not part of the released full-pack artifact.

NeurIPS Paper Checklist

1. Claims

Question: Do the main claims made in the abstract and introduction accurately reflect the paper’s contributions and scope?

Answer: [\[Yes\]](#)

Justification: The abstract and introduction state the paper’s scope as introducing an execution-based benchmark for web-enabled PCB reconstruction agents, documenting its harness protocol and external-I/O oracle, validating the scoring artifact with canaries, and reporting initial harness baselines. The paper explicitly excludes hidden-answer circuit invention, fabrication readiness, regulatory certification, and vendor-accurate electrical validation.

Guidelines:

- The answer [\[N/A\]](#) means that the abstract and introduction do not include the claims made in the paper.
- The abstract and/or introduction should clearly state the claims made, including the contributions made in the paper and important assumptions and limitations. A [\[No\]](#) or [\[N/A\]](#) answer to this question will not be perceived well by the reviewers.
- The claims made should match theoretical and experimental results, and reflect how much the results can be expected to generalize to other settings.

- It is fine to include aspirational goals as motivation as long as it is clear that these goals are not attained by the paper.

2. Limitations

Question: Does the paper discuss the limitations of the work performed by the authors?

Answer: [Yes]

Justification: The Limitations section discusses public-source task exposure, task-set size, oracle fidelity, and upstream license constraints. The benchmark overview and access-policy calibration sections also clarify that the default baselines are web-enabled and should not be interpreted as closed-book PCB invention.

Guidelines:

- The answer [N/A] means that the paper has no limitation while the answer [No] means that the paper has limitations, but those are not discussed in the paper.
- The authors are encouraged to create a separate “Limitations” section in their paper.
- The paper should point out any strong assumptions and how robust the results are to violations of these assumptions.
- The authors should reflect on the scope of the claims made, e.g., if the approach was only tested on a few datasets or with a few runs.
- The authors should reflect on the factors that influence the performance of the approach.
- The authors should discuss the computational efficiency of the proposed algorithms and how they scale with dataset size.
- If applicable, the authors should discuss possible limitations of their approach to address problems of privacy and fairness.
- While the authors might fear that complete honesty about limitations might be used by reviewers as grounds for rejection, a worse outcome might be that reviewers discover limitations that are not acknowledged in the paper.

3. Theory assumptions and proofs

Question: For each theoretical result, does the paper provide the full set of assumptions and a complete (and correct) proof?

Answer: [N/A]

Justification: The paper is a benchmark and empirical evaluation paper and does not include theoretical results or proofs.

Guidelines:

- The answer [N/A] means that the paper does not include theoretical results.
- All the theorems, formulas, and proofs in the paper should be numbered and cross-referenced.
- All assumptions should be clearly stated or referenced in the statement of any theorems.
- The proofs can either appear in the main paper or the supplemental material, but if they appear in the supplemental material, the authors are encouraged to provide a short proof sketch to provide intuition.
- Inversely, any informal proof provided in the core of the paper should be complemented by formal proofs provided in appendix or supplemental material.
- Theorems and Lemmas that the proof relies upon should be properly referenced.

4. Experimental result reproducibility

Question: Does the paper fully disclose all the information needed to reproduce the main experimental results of the paper to the extent that it affects the main claims and/or conclusions of the paper (regardless of whether the code and data are provided or not)?

Answer: [Yes]

Justification: The paper reports the task-pack revision, harness protocol, scoring formula, runtime and grading artifact split, provenance dataset, source run identifiers, and regrade procedure. The code, released task artifacts, grading artifacts, and saved final projects are public or described through the release and quarantine policy.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- If the paper includes experiments, a [No] answer to this question will not be perceived well by the reviewers: Making the paper reproducible is important, regardless of whether the code and data are provided or not.
- If the contribution is a dataset and/or model, the authors should describe the steps taken to make their results reproducible or verifiable.
- Depending on the contribution, reproducibility can be accomplished in various ways.
- While NeurIPS does not require releasing code, the conference does require all submissions to provide some reasonable avenue for reproducibility, which may depend on the nature of the contribution.

5. Open access to data and code

Question: Does the paper provide open access to the data and code, with sufficient instructions to faithfully reproduce the main experimental results, as described in supplemental material?

Answer: [Yes]

Justification: Appendix A identifies the source repository, Hugging Face task dataset, task-pack revision, Croissant metadata, and public provenance dataset. Appendix F describes the license triage policy and states that five upstream-license-risk candidate tasks are quarantined from the released full-pack artifact.

Guidelines:

- The answer [N/A] means that paper does not include experiments requiring code.
- Please see the NeurIPS code and data submission guidelines (<https://neurips.cc/public/guides/CodeSubmissionPolicy>) for more details.
- While we encourage the release of code and data, we understand that this might not be possible, so [No] is an acceptable answer.
- The instructions should contain the exact command and environment needed to run to reproduce the results.
- The authors should provide instructions on data access and preparation.
- The authors should provide scripts to reproduce all experimental results for the new proposed method and baselines.
- At submission time, to preserve anonymity, the authors should release anonymized versions, if applicable.
- Providing as much information as possible in supplemental material is recommended, but including URLs to data and code is permitted.

6. Experimental setting/details

Question: Does the paper specify all the training and test details (e.g., data splits, hyperparameters, how they were chosen, type of optimizer) necessary to understand the results?

Answer: [Yes]

Justification: The benchmark overview, harness protocol, scoring oracle, artifact validation, and experiments sections describe the task interface, Dockerized tool environment, model and harness configurations, score definition, build-success definition, task count, validation controls, and access-policy separation.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The experimental setting should be presented in the core of the paper to a level of detail that is necessary to appreciate the results and make sense of them.
- The full details can be provided either with the code, in appendix, or as supplemental material.

7. Experiment statistical significance

Question: Does the paper report error bars suitably and correctly defined or other appropriate information about the statistical significance of the experiments?

Answer: [Yes]

Justification: The paper states that the reported results are complete released-task sweeps rather than repeated independent sampling campaigns. It therefore does not claim statistically significant pairwise model rankings. The supported conclusions are artifact-level and qualitative: the benchmark is not saturated by current evaluated harnesses, build success diverges from functional score, and access policy affects the evaluated harnesses under the reported protocol.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The authors should answer [Yes] if the results are accompanied by error bars, confidence intervals, or statistical significance tests, at least for the experiments that support the main claims of the paper.
- The factors of variability that the error bars are capturing should be clearly stated.
- The method for calculating the error bars should be explained.
- The assumptions made should be given.
- It should be clear whether the error bar is the standard deviation or the standard error of the mean.
- For asymmetric distributions, the authors should be careful not to show symmetric error bars that would yield results that are out of range.
- If error bars are reported in tables or plots, the authors should explain how they were calculated and reference the corresponding figures or tables.

8. Experiments compute resources

Question: For each experiment, does the paper provide sufficient information on the computer resources (type of compute workers, memory, time of execution) needed to reproduce the experiments?

Answer: [Yes]

Justification: The paper specifies the Dockerized KiCad, ngspice, Python, and Node environment, records run identifiers and provenance, and reports cost and provenance-size accounting for the main Codex campaign in Appendix E. Saved run provenance includes additional execution details for reproduced runs.

Guidelines:

- The answer [N/A] means that the paper does not include experiments.
- The paper should indicate the type of compute workers CPU or GPU, internal cluster, or cloud provider, including relevant memory and storage.
- The paper should provide the amount of compute required for each of the individual experimental runs as well as estimate the total compute.
- The paper should disclose whether the full research project required more compute than the experiments reported in the paper.

9. Code of ethics

Question: Does the research conducted in the paper conform, in every respect, with the NeurIPS Code of Ethics <https://neurips.cc/public/EthicsGuidelines?>

Answer: [Yes]

Justification: The work is a benchmark and evaluation study without human subjects or deception. The paper discusses limitations, release constraints, and the risk of overtrust in generated hardware artifacts.

Guidelines:

- The answer [N/A] means that the authors have not reviewed the NeurIPS Code of Ethics.
- If the authors answer [No], they should explain the special circumstances that require a deviation from the Code of Ethics.
- The authors should make sure to preserve anonymity.

10. Broader impacts

Question: Does the paper discuss both potential positive societal impacts and negative societal impacts of the work performed?

Answer: [Yes]

Justification: The paper frames the benchmark as a tool for measuring EDA-agent reliability and explicitly states that scores are not evidence of manufacturing readiness, safety, EMC compliance, regulatory compliance, or production readiness. The main negative impact is overtrust in generated hardware artifacts. The mitigation is conservative scoring, validation controls, and explicit scope limits.

Guidelines:

- The answer [N/A] means that there is no societal impact of the work performed.
- If the authors answer [N/A] or [No], they should explain why their work has no societal impact or why the paper does not address societal impact.
- Examples of negative societal impacts include potential malicious or unintended uses, fairness considerations, privacy considerations, and security considerations.
- The conference expects that many papers will be foundational research and not tied to particular applications, let alone deployments.
- The authors should consider possible harms that could arise when the technology is being used as intended and functioning correctly, when it is being used as intended but gives incorrect results, and from misuse.
- If there are negative societal impacts, the authors could also discuss possible mitigation strategies.

11. Safeguards

Question: Does the paper describe safeguards that have been put in place for responsible release of data or models that have a high risk for misuse (e.g., pre-trained language models, image generators, or scraped datasets)?

Answer: [N/A]

Justification: The paper releases benchmark tasks, grading assets, and evaluation code rather than a new high-risk generative model or scraped personal or sensitive dataset. Upstream-license-risk hardware snapshots are quarantined from the released full-pack artifact until permission or acquisition-script replacement is available.

Guidelines:

- The answer [N/A] means that the paper poses no such risks.
- Released models that have a high risk for misuse or dual-use should be released with necessary safeguards to allow for controlled use of the model.
- Datasets that have been scraped from the Internet could pose safety risks.
- We recognize that providing effective safeguards is challenging, and many papers do not require this, but we encourage authors to take this into account and make a best faith effort.

12. Licenses for existing assets

Question: Are the creators or original owners of assets (e.g., code, data, models), used in the paper, properly credited and are the license and terms of use explicitly mentioned and properly respected?

Answer: [Yes]

Justification: Appendix F states the license triage policy and identifies the five quarantined candidate tasks. The repository includes LICENSES.md, task-pack license notes, and source provenance metadata.

Guidelines:

- The answer [N/A] means that the paper does not use existing assets.
- The authors should cite the original paper that produced the code package or dataset.
- The authors should state which version of the asset is used and, if possible, include a URL.
- The name of the license should be included for each asset.

- For scraped data from a particular source, the copyright and terms of service of that source should be provided.
- If assets are released, the license, copyright information, and terms of use in the package should be provided.
- For existing datasets that are re-packaged, both the original license and the license of the derived asset should be provided.
- If this information is not available online, the authors are encouraged to reach out to the asset's creators.

13. New assets

Question: Are new assets introduced in the paper well documented and is the documentation provided alongside the assets?

Answer: [\[Yes\]](#)

Justification: The paper documents the task taxonomy, runtime and full-pack split, scoring oracle, validation controls, provenance structure, Croissant metadata, artifact availability, and release policy. The repository and dataset contain detailed task and release documentation.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not release new assets.
- Researchers should communicate the details of the dataset/code/model as part of their submissions via structured templates.
- The paper should discuss whether and how consent was obtained from people whose asset is used.
- At submission time, remember to anonymize your assets if applicable.

14. Crowdsourcing and research with human subjects

Question: For crowdsourcing experiments and research with human subjects, does the paper include the full text of instructions given to participants and screenshots, if applicable, as well as details about compensation (if any)?

Answer: [\[N/A\]](#)

Justification: The paper does not involve crowdsourcing or research with human subjects.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not involve crowdsourcing nor research with human subjects.
- Including this information in the supplemental material is fine, but if the main contribution of the paper involves human subjects, then as much detail as possible should be included in the main paper.
- According to the NeurIPS Code of Ethics, workers involved in data collection, curation, or other labor should be paid at least the minimum wage in the country of the data collector.

15. Institutional review board (IRB) approvals or equivalent for research with human subjects

Question: Does the paper describe potential risks incurred by study participants, whether such risks were disclosed to the subjects, and whether Institutional Review Board (IRB) approvals (or an equivalent approval/review based on the requirements of your country or institution) were obtained?

Answer: [\[N/A\]](#)

Justification: The paper does not involve human-subjects research.

Guidelines:

- The answer [\[N/A\]](#) means that the paper does not involve crowdsourcing nor research with human subjects.
- Depending on the country in which research is conducted, IRB approval or equivalent may be required for any human subjects research.

- 743 • We recognize that the procedures for this may vary significantly between institutions
744 and locations, and we expect authors to adhere to the NeurIPS Code of Ethics and the
745 guidelines for their institution.
746 • For initial submissions, do not include any information that would break anonymity, if
747 applicable, such as the institution conducting the review.

748 **16. Declaration of LLM usage**

749 Question: Does the paper describe the usage of LLMs if it is an important, original, or
750 non-standard component of the core methods in this research? Note that if the LLM is used
751 only for writing, editing, or formatting purposes and does *not* impact the core methodology,
752 scientific rigor, or originality of the research, declaration is not required.

753 Answer: [N/A]

754 Justification: LLMs are not important, original, or non-standard components of the core
755 method development in this research.

756 Guidelines:

- 757 • The answer [N/A] means that the core method development in this research does not
758 involve LLMs as any important, original, or non-standard components.
759 • Please refer to our LLM policy in the NeurIPS handbook for what should or should not
760 be described.